

PROG7311 – 3A
Summative Assessment

Student Number:	St10433940
Programme Code:	BCAD0701
Module Lecturer:	Mortda Adam
Module Code:	PROG7311
Date of Submission:	07-06-2026

I hereby declare that I did not plagiarise the content of this assignment and that this is my own work. I also declare that no use of GenAI was used in the creation of this submission (unless explicitly permitted and declared), excluding grammar and spell checkers. I consent to the lecturer assessing our work using an AI detection tool to ensure the integrity of our work.

Assignment submitted via SafeAssign: ☒ (Tick the Box)

Table of Contents

1	Question 1.....	1
---	-----------------	---

Information Systems 3A

Student Number:	10433940
Programme Code:	BCAD0701
Module Lecturer:	Samkelo Magwenyane
Module Code:	INSY7311
Date of Submission:	23-03-2026

I hereby declare that I did not plagiarise the content of this assignment and that this is my own work. I also declare that no use of GenAI was used in the creation of this submission (unless explicitly permitted and declared), excluding grammar and spell checkers. I consent to the lecturer assessing our work using an AI detection tool to ensure the integrity of our work.

Assignment submitted via SafeAssign: ☒ (Tick the Box)

Table of Contents

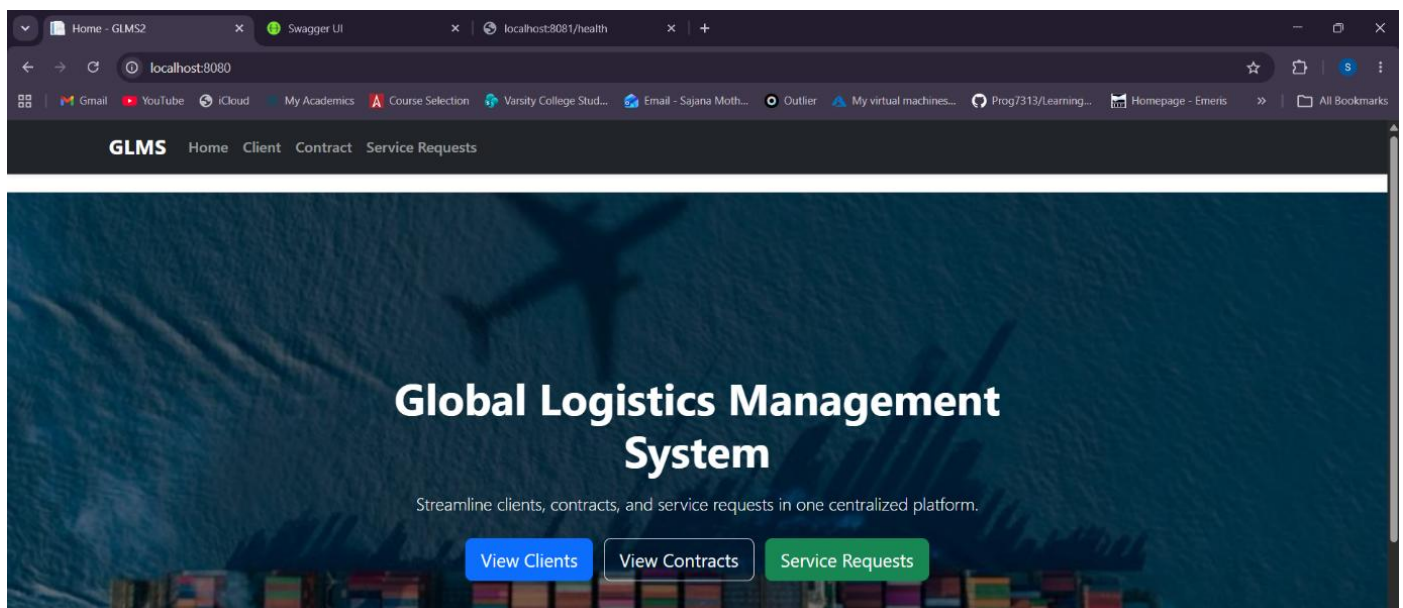
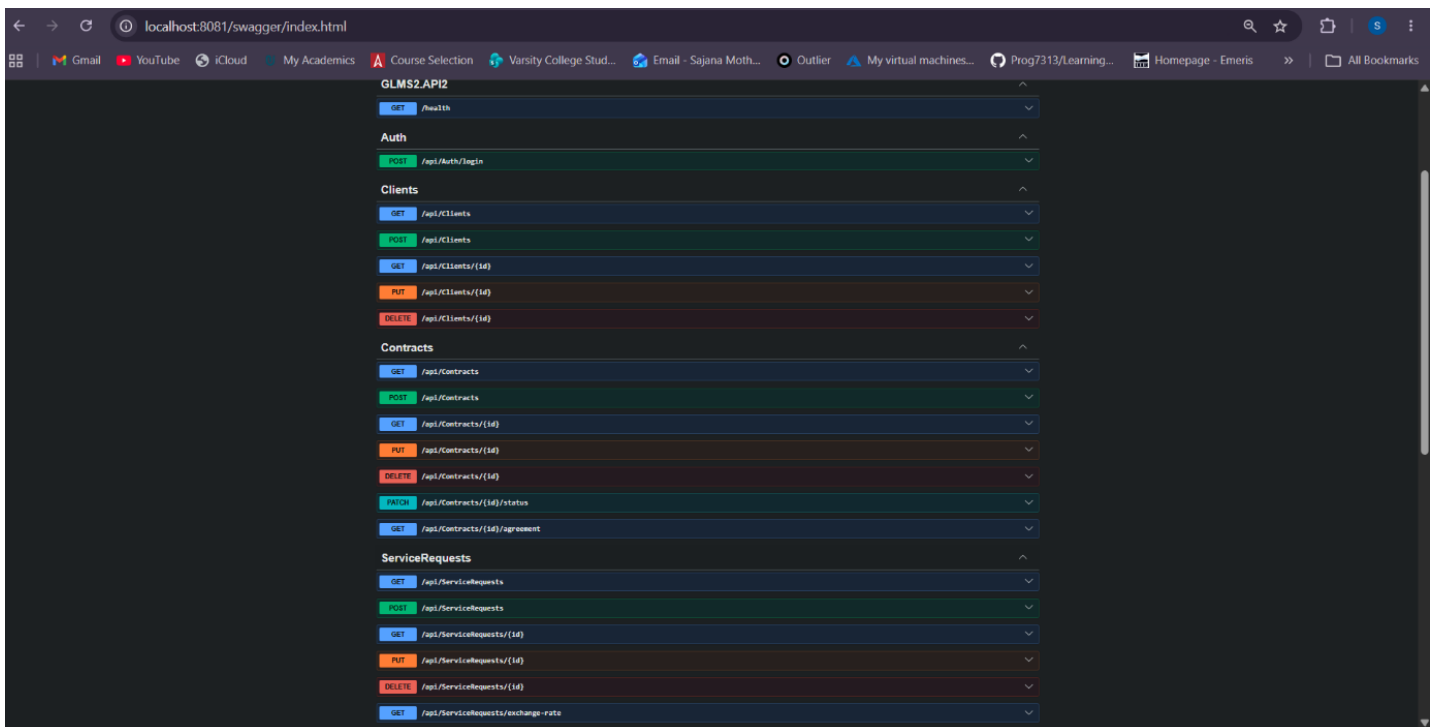
1	Question 1.....	Error! Bookmark not defined.
2	Question 2.....	Error! Bookmark not defined.
3	Question 3.....	Error! Bookmark not defined.
4	Question 4.....	Error! Bookmark not defined.
5	Question 5.....	Error! Bookmark not defined.
6	References.....	Error! Bookmark not defined.

1 QUESTION 1

GitHub: <https://github.com/EMKNPM/programming-3a-prog7311-part-3-Sajana2101>

YouTube: <https://youtu.be/mpi0PVBhEdI>

Screen Shots



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\sajan> cd C:\Users\sajan\source\repos\GLMS2
PS C:\Users\sajan\source\repos\GLMS2> docker compose ps
NAME                IMAGE                                     COMMAND                                SERVICE        CREATED        STATUS        PORTS
glms-backend-api    glms-part3-glms-backend-api            "dotnet GLMS2.API2.d..."           glms-backend-api About an hour ago Up About an hour 0.0.0.0
:8081->8080/tcp, [::]:8081->8080/tcp
glms-frontend-web   glms-part3-glms-frontend-web           "dotnet GLMS2.dll"                  glms-frontend-web About an hour ago Up About an hour 0.0.0.0
:8080->8080/tcp, [::]:8080->8080/tcp
sql-server-db       mcr.microsoft.com/mssql/server:2022-latest "/opt/mssql/bin/laun..."          sql-server-db    About an hour ago Up About an hour 0.0.0.0
:14333->1433/tcp, [::]:14333->1433/tcp
PS C:\Users\sajan\source\repos\GLMS2>
```

docker desktop PERSONAL

Search Ctrl+K

Containers Give feedback

Container CPU usage: 1.48% / 800% (8 CPUs available) Container memory usage: 1.14GB / 7.45GB [Show charts](#)

Search

☐ Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (%)	Disk rea	Actions
☐	glms-part3	-	-	-	1.48%	1.14GB / 22.89GI	14.96%	1.33GB	
☐	sql-server-db	4a8ce7d34676	mssql/serv	14333:1433	1.45%	921.1MB / 7.63G	11.79%	1.32GB	
☐	glms-backen	a8f9aae8f03e	glms-part3	8081:8080	0.01%	147.2MB / 7.63G	1.88%	5.86MB	
☐	glms-fronten	212b0e1906e5	glms-part3	8080:8080	0.02%	101MB / 7.63GB	1.29%	2.99MB	

Showing 4 items

Engine running | RAM 2.14 GB | CPU 0.50% | Disk: 6.16 GB used (limit 1006.85 GB) Terminal v4.76.0

Test Explorer

Test run finished: 16 Tests (16 Passed, 0 Failed, 0 Skipped) run in 6,4 sec 0 Warnings 0 Errors

Test	Duration	Tr
GLMS2.Test (16)	7,9 sec	
GLMS2.Test.IntegrationTests (6)	6 sec	
ApiIntegrationTests (6)	6 sec	
CreateClient_CreateContract_U...	5,7 sec	
CreateContract_WithoutToken_...	54 ms	
GetContracts_ReturnsOk_AndJs...	133 ms	
GetExchangeRate_ReturnsPositi...	69 ms	
Login_WithInvalidCredentials_R...	42 ms	
Login_WithValidCredentials_Ret...	18 ms	
GLMS2.Tests (10)	1,8 sec	
CurrencyServiceTests (4)	23 ms	
FileServiceTests (3)	14 ms	
ServiceRequestServiceTests (3)	1,8 sec	

Run | Debug | Profile

2 QUESTION 2

Automated Testing

Automated Testing, also known as Automation Testing, is an approach that tests software through a series of test cases that are executed using scripts and tools instead of manual efforts (GeeksforGeeks, 2019). Automated testing is an essential part of any software project, more specifically in terms of a Continuous Integration/Continuous Delivery (CI/CD) pipeline (Verma, 2018). Automation testing is essential to a successful CI/CD pipeline because automation testing ensures that small changes can be easily handled, tests are faster, consistency, and agility (Verma, 2018). CI/CD involves many small updates to the code, and automated testing ensures that small changes are checked quickly without a developer team having to test everything each time (Verma, 2018) manually. CI/CD needs to be fast, and automated tests run faster than manual testing, especially since multiple tests can be run simultaneously, thereby speeding up the CI/CD process (Ferris, 2023). A CI/CD pipeline must behave the same way each time, and automated testing helps ensure that any mistakes or differences that may happen with manual testing can be avoided, thus ensuring consistency (Verma, 2018). When tools, requirements, or frameworks change, teams need to adapt quickly; automated tests make adjusting the pipeline easier, thus ensuring agility (Ferris, 2023). Automated testing can ensure bugs do not reach production for a number of reasons (Chukwube, 2025). Automated tests run with every commit, meaning that each change to the code is checked immediately before it moves further in the pipeline (Chukwube, 2025). This ensures that bugs can be caught early instead of only finding them after release. Automated testing quickly shows whether a new change has broken something (Chukwube, 2025). With continuous testing running across the entire workflow, bugs are checked for throughout development, not just before release (Chukwube, 2025). Additionally, since testing happens at every milestone, bugs can be prevented because software is repeatedly at different stages (Chukwube, 2025).

Containerization

The “it works on my machine” problem is a common software developer problem where a developer has built a project or written code that works perfectly on their local machines but breaks downstream CI pipeline or production (Milligan, 2023). A Docker container image is a lightweight executable package (image) of software that includes everything required to run an application (Docker, 2025). This means that any application using Docker does not rely on what is installed on the developer’s computer. Docker can solve the “it works on my machine” problem through consistency (Ozcan, 2023). Docker keeps application behaviour the same in development, testing, and production because everything the app needs is inside the container (Ozcan, 2023). Instead of developers having different software installed on their laptops, Docker gives every developer the same environment to run and develop the application (Siddharth, 2025). Thus, ensuring consistency across the developer environment. According to Ozcan (2023), “Docker ensures that the test environment remains consistent throughout the entire testing lifecycle.” The same image that was used during development can be used for testing and production, reducing the risk of any

environment-related issues (Ozcan, 2023). Similarly, Docker ensures that applications run the same way in development and testing as it would in production (GeeksforGeeks, 2024). Docker, all in all, bundles an application with all dependencies, libraries, tools, and settings so that the app does not depend on what has been installed on a specific machine, eliminating the “it works on my machine” problem because the same containerised setup can be used across all environments (Ozcan, 2023; GeeksforGeeks, 2024; Docker, 2025).

3 REFERENCES

- Chukwube, M., 2025. *Continuous Testing Strategies That Actually Prevent Production Bugs - DevOps.com*. [online] DevOps. Available at: <<https://devops.com/continuous-testing-strategies-that-actually-prevent-production-bugs/>> [Accessed June 2026].
- Docker, 2025. *What is a Container?* [online] Docker. Available at: <<https://www.docker.com/resources/what-container/>> [Accessed June 2026].
- Ferris, M., 2023. *How Does Automated Testing For CI/CD Help Improve Software Quality and Delivery*. [online] Qameta.io. Available at: <<https://qameta.io/blog/automated-testing-for-ci-cd>> [Accessed June 2026].
- GeeksforGeeks, 2019. *Automation Testing Software Testing*. [online] GeeksforGeeks. Available at: <<https://www.geeksforgeeks.org/software-testing/automation-testing-software-testing/>> [Accessed June 2026].
- GeeksforGeeks, 2024. *Building Docker Development Environments: Benefits and Challenges*. [online] GeeksforGeeks. Available at: <<https://www.geeksforgeeks.org/devops/docker-development-environments-benefits-challenges/>> [Accessed 7 June 2026].
- Milligan, M., 2023. *'It works on my machine' explained*. [online] Coder. Available at: <<https://coder.com/blog/it-works-on-my-machine-explained>> [Accessed 7 June 2026].
- Ozcan, N.T., 2023. *Docker for Software Test Engineers: Streamlining Test Environments with Containerization*. [online] Medium. Available at: <<https://medium.com/@nevzattalhaozcan/docker-for-software-test-engineers-streamlining-test-environments-with-containerization-5cb57897b34a>> [Accessed June 2026].
- Siddharth, 2025. *Using Containerization (Docker) for Consistent Dev Environments in Scrum*. [online] Agile Seekers. Available at: <<https://agileseekers.com/blog/using-containerization-docker-for-consistent-dev-environments-in-scrum>> [Accessed June 2026].

Verma, A., 2018. *Why Test Automation is Essential for CI/CD*. [online] Functionize. Available at: <<https://www.functionize.com/blog/why-test-automation-is-essential-for-ci-cd>> [Accessed 2026].